# A Practical Guide to Word2vec

TOPIC -- WORD2VEC

$$J(\theta) = -\frac{1}{T}\sum_{t=1}^{T}\sum_{j\neq 0} \log p(w_{t+j} \mid w_t)$$

-- 01 --

Approach Word2vec is a group of related models that are used to produce word embeddings. These models are shallow, two-layer neural networks that are trained to reconstruct linguistic contexts of words. Word2vec takes as its input a large corpus of text and produces a mapping of the set of words to a vector space, typically of several hundred dimensions, with each unique word in the corpus being assigned a vector in the space. Word2vec can use either of two model architectures to produce these distributed representations of words: continuous bag of words (CBOW) or continuously sliding skip-gram. In both architectures, word2vec considers both individual words and a sliding context window as it iterates over the corpus. The CBOW can be viewed as a 'fill in the blank' task, where the word embedding represents the way the word influences the relative probabilities of other words in the context window. Words which are semantically similar should influence these probabilities in similar ways, because semantically similar words should be used in similar contexts. The order of context words does not influence prediction (bag of words assumption). In the continuous skip-gram architecture, the model uses the current word to predict the surrounding window of context words. The skip-gram architecture weighs nearby context words more heavily than more distant context words. According to the authors' note, CBOW is faster while skip-gram does a better job for infrequent words. After the model is trained, the learned word embeddings are positioned in the vector space such that words that share common contexts in the corpus — that is, words that are semantically and syntactically similar — are located close to one another in the space. More dissimilar words are located farther from one another in the space.

-- 02 --

History During the 1980s, there were some early attempts at using neural networks to represent words and concepts as vectors. In 2010, Tomáš Mikolov (then at Brno University of Technology) with co-authors applied a simple recurrent neural network with a single hidden layer to language modelling. Word2vec was created, patented, and published in 2013 by a team of

researchers led by Mikolov at Google over two papers. The original paper was rejected by reviewers for ICLR conference 2013. It also took months for the code to be approved for open-sourcing. Other researchers helped analyse and explain the algorithm. Embedding vectors created using the Word2vec algorithm have some advantages compared to earlier algorithms such as those using n-grams and latent semantic analysis. GloVe was developed by a team at Stanford specifically as a competitor, and the original paper noted multiple improvements of GloVe over word2vec. Mikolov argued that the comparison was unfair as GloVe was trained on more data, and that the fastText project showed that word2vec is superior when trained on the same data. As of 2022, the straight Word2vec approach was described as "dated". Transformer-based models, such as ELMo and BERT, which add multiple neural-network attention layers on top of a word embedding model similar to Word2vec, have come to be regarded as the state of the art in natural language processing.

-- 03 --

Context window The size of the context window determines how many words before and after a given word are included as context words of the given word. According to the authors' note, the recommended value is 10 for skip-gram and 5 for CBOW.

-- 04 --

Word2vec is a technique in natural language processing for obtaining vector representations of words. These vectors capture information about the meaning of the word based on the surrounding words. The word2vec algorithm estimates these representations by modeling text in a large corpus. Once trained, such a model can detect synonymous words or suggest additional words for a partial sentence. Word2vec was developed by Tomáš Mikolov, Kai Chen, Greg Corrado, Ilya Sutskever and Jeff Dean at Google, and published in 2013. Word2vec represents a word as a high-dimension vector of numbers which capture relationships between words. In particular, words which appear in similar contexts are mapped to vectors which are nearby as measured by cosine similarity. This indicates the level of semantic similarity between the words, so for example the vectors for walk and ran are nearby, as are those for "but" and "however", and "Berlin" and "Germany".

-- 05 --

Analysis The reasons for successful word embedding learning in the word2vec framework are poorly understood. Goldberg and Levy point out that the word2vec objective function causes words that occur in similar contexts to have

similar embeddings (as measured by cosine similarity) and note that this is in line with J. R. Firth's distributional hypothesis. However, they note that this explanation is "very hand-wavy" and argue that a more formal explanation would be preferable. Levy et al. (2015) show that much of the superior performance of word2vec or similar embeddings in downstream tasks is not a result of the models per se, but of the choice of specific hyperparameters. Transferring these hyperparameters to more 'traditional' approaches yields similar performances in downstream tasks. Arora et al. (2016) explain word2vec and related algorithms as performing inference for a simple generative model for text, which involves a random walk generation process based upon loglinear topic model. They use this to explain some properties of word embeddings, including their use to solve analogies.

**-- 06 --**

The idea of CBOW is to represent each word with a vector, such that it is possible to predict a word using the sum of the vectors of its neighbors. Specifically, for each word w i {\displaystyle w_{i}} in the corpus, the one-hot encoding of the word is used as the input to the neural network. The output of the neural network is a probability distribution over the dictionary, representing a prediction of individual words in the neighborhood of w i {\displaystyle w_{i}} . The objective of training is to maximize ∑ i ln ■ Pr ( w i ■ w i + j : j ∈ N ) {\displaystyle \sum _{i}\ln \Pr(w_{i}\mid w_{i+j}\colon j\in N)} where N {\displaystyle N} is a set of (non-zero) indices representing the relative locations of nearby words considered to be in w i {\displaystyle w_{i}} 's neighborhood. For example, if we want each word in the corpus to be predicted by every other word in a small span of 4 words. The set of relative indexes of neighbor words will be: N = { – 2 , – 1 , + 1 , + 2 } {\displaystyle N=\{-2,-1,+1,+2\}} , and the objective is to maximize ∑ i ln ■ Pr ( w i ■ w i – 2 , w i – 1 , w i + 1 , w i + 2 ) {\displaystyle \sum _{i}\ln \Pr(w_{i}\mid w_{i-2},w_{i-1},w_{i+1},w_{i+2})} . In standard bag-of-words, a word's context is represented by a word-count (aka a word histogram) of its neighboring words. For example, the "sat" in "the cat sat on the mat" is represented as {"the": 2, "cat": 1, "on": 1}. Note that the last word "mat" is not used to represent "sat", because it is outside the neighborhood N = { – 2 , – 1 , + 1 , + 2 } {\displaystyle N=\{-2,-1,+1,+2\}} . In continuous bag-of-words, the histogram is multiplied by a matrix V {\displaystyle V} to obtain a continuous representation of the word's context. The matrix V {\displaystyle V} is also called a dictionary. Its columns are the word vectors. It has D {\displaystyle D} columns, where D {\displaystyle D} is the size of the dictionary. Let d {\displaystyle d} be the length of each word vector. We have V ∈ R d × D {\displaystyle V\in \mathbb {R} ^{d\times D}} . For example, multiplying the word histogram {"the": 2, "cat": 1, "on": 1} with V {\displaystyle V} , we obtain 2 v the + v cat + v on {\displaystyle

2v_{\text{the}}+v_{\text{cat}}+v_{\text{on}}} . This is then multiplied with another matrix V ′ {\displaystyle V'} of shape R D × d {\displaystyle \mathbb {R} ^{D\times d}} . Each row of it is a word vector v ′ {\displaystyle v'} . This results in a vector of length D {\displaystyle D} , one entry per dictionary entry. Then, apply the softmax to obtain a probability distribution over the dictionary. This system can be visualized as a neural network, similar in spirit to an autoencoder, of architecture linear-linear-softmax, as depicted in the diagram. The system is trained by gradient descent to minimize the cross-entropy loss. In full formula, the cross-entropy loss is: – ∑ i ln ■ e v w i ′ · ( ∑ j ∈ N v w j + i ) ∑ w ′ e v w ′ ′ · ( ∑ j ∈ N v w j + i ) {\displaystyle -\sum _{i}\ln {\frac {e^{v_{w_{i}}'\cdot (\sum _{j\in N}v_{w_{j+i}})}}{\sum _{w'}e^{v_{w'}'\cdot (\sum _{j\in N}v_{w_{j+i}})}}}} where the outer summation ∑ i {\displaystyle \sum _{i}} is over the words in a corpus, the quantity ∑ j ∈ N v w j + i {\displaystyle \sum _{j\in N}v_{w_{j+i}}} is the sum of a word's neighbors' vectors, etc. Once such a system is trained, we have two trained matrices V , V ′ {\displaystyle V,V'} . Either the column vectors of V {\displaystyle V} or the row vectors of V ′ {\displaystyle V'} can serve as the dictionary. For example, the word "sat" can be represented as either the "sat"-th column of V {\displaystyle V} or the "sat"-th row of V ′ {\displaystyle V'} . It is also possible to simply define V ′ = V ■ {\displaystyle V'=V^{\top }} , in which case there would no longer be a choice.